



Companion to: `computer-vision-notebook_B0.ipynb` + `CV_Notebook_Math_Reference.pdf`

Table of Contents

1. Project Overview
2. Environment & Imports
3. Configuration & Hyperparameters
4. Two-Stage Inference Pipeline
5. Custom Augmentation Classes
6. Image Transform Pipeline
7. Dataset Class: RimDataset
8. Data Loading: Splitting, Sampling & DataLoaders
9. Attention Modules
10. The Ensemble Model: EnsembleModelV6
11. Loss Functions
12. EMA: Exponential Moving Average
13. MixUp & CutMix Augmentation
14. Test-Time Augmentation (TTA)
15. Validation & Threshold Sweep
16. Batch Normalisation Recalibration
17. Platt Scaling (Probability Calibration)
18. Three-Phase Training Loop
19. AMP: Automatic Mixed Precision
20. Full Math Reference Index

1. Project Overview

This notebook solves a **binary industrial defect detection problem**: classifying bottle rims as either *good* (label 0) or *faulty* (label 1) from camera images. It was built for the **1st Krones Vision AI Challenge** on Kaggle.

The core challenge has two layers of difficulty:

- **Class imbalance:** ~90% good rims → naive model gets 90% accuracy while catching zero defects.
- **Subtle defects:** small chips, hairline cracks, or contamination confined to 1% of the image.

The solution is a two-stage pipeline backed by a three-backbone ensemble with specialized loss functions, attention mechanisms, and a multi-phase training schedule.

2. Environment & Imports

```
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "max_split_size_mb:64"
```

The CUDA allocator is configured before any CUDA call to limit fragmentation on Kaggle's shared GPU environment. Optional libraries (augmentations) are guarded with try/except. ImageNet constants are defined once and reused everywhere.

$$x'_c = (x_c - \mu_c) / \sigma_c$$

3. Configuration & Hyperparameters

Centralized in `argparse`. Key hyperparameters:

Flag	Default	Role
--batch	16	Mini-batch size per GPU
--accum-steps	8	Effective batch = 128
--lr	5e-5	Base learning rate
--img-size	260	Input resolution
--focal-gamma	2.0	Focal Loss exponent γ
--ema-decay	0.9998	EMA shadow decay δ
--mixup-alpha	0.05	Beta distribution parameter
--label-smooth	0.02	Label smoothing ϵ
--dropout-rate	0.35	Dropout probability
--grad-clip	0.5	Gradient norm clipping

Stage 1 config: threshold 0.70, image size 384px, 15 epochs, batch 32.

4. Two-Stage Inference Pipeline

Stage 1 (EfficientNet-B0): lightweight gate that filters obvious defects. High-confidence faulty images are rejected directly; ambiguous ones pass to Stage 2.

Stage 2 (EnsembleV6): three-backbone ensemble (EfficientNet-B3, RegNetY-8GF, ConvNeXt-Small) that runs only on uncertain images, maximising throughput.

5. Custom Augmentation Classes

All inherit from `A.ImageOnlyTransform` and simulate real industrial conditions.

- **RadialBlur:** diagonal motion blur via convolution kernel.
- **SpecularHighlight:** 2D Gaussian bright spot (metal glare).
- **ScratchSimulation:** near-white thin lines to distinguish transport scratches from real cracks.
- **SectorMask:** pie-slice occlusion filled with mean color.
- **RimSpecificAug:** vignette, ring light, fixture shadow.

6. Image Transform Pipeline

`_make_train_transform` uses a curriculum: light augmentation (rotations, flips, noise) for early phases, heavy augmentation (ElasticTransform, CLAHE, CoarseDropout + custom simulators) for phase 3. Validation uses only resize + normalisation.

7. Dataset Class: RimDataset

Supports RAM caching (`cache=True`) to avoid repeated disk reads. Training does not cache because each augmented view is different. Uses albumentations if available, otherwise falls back to torchvision.

8. Data Loading: Splitting, Sampling & DataLoaders

Stratified split preserves class ratio. Two balancing strategies:

- **Oversampling:** duplicate minority samples to reach 1:1 ratio.
- **WeightedRandomSampler:** probability inversely proportional to class size.

DataLoader uses `pin_memory=True` , `persistent_workers=True` , `prefetch_factor=4` for maximum throughput.

9. Attention Modules

CBAM: sequential channel and spatial attention. Channel branch: $\sigma(\text{MLP}(\text{AvgPool}) + \text{MLP}(\text{MaxPool}))$. Spatial branch: $\sigma(\text{Conv}7\times7([\text{AvgPool}_c; \text{MaxPool}_c]))$.

AttentionPool2d: replaces global average pooling. Uses learned 1×1 convolution + softmax to weight spatial locations, preventing small defects from being diluted.

10. The Ensemble Model: EnsembleModelV6

Three backbones (EfficientNet-B3, RegNetY-8GF, ConvNeXt-Small) each produce a 320-dimensional representation after CBAM + AttentionPool + projection. A multi-head self-attention layer (8 heads) lets backbones attend to each other, followed by gated fusion (sigmoid-gated linear layer) and a shared classification head with two outputs (binary logit + severity regression).

11. Loss Functions

- **FocalLossWithLogits** ($\gamma=2.0$): down-weights easy examples.
- **AsymmetricLoss** ($\gamma^+=1, \gamma^-=4$): aggressively down-weights easy negatives.
- **CombinedLoss**: $0.6 \cdot \text{FL} + 0.4 \cdot \text{ASL} + 0.25 \cdot \text{MSE}(\sigma(\text{sev_logits}), y)$.

12. EMA: Exponential Moving Average

Maintains a shadow model with smoothed weights. At inference, the EMA model generalises better. Warm-up decay ensures fast initial convergence.

$$\theta_{\text{shadow}} \leftarrow d \cdot \theta_{\text{shadow}} + (1-d) \cdot \theta_{\text{model}}, \quad d = \min(\delta, (1+t)/(10+t))$$

13. MixUp & CutMix Augmentation

Apply with 60% probability, half the time MixUp (linear blend), half CutMix (paste rectangular patch). Labels become soft (lambda blend), seamlessly supporting focal loss.

14. Test-Time Augmentation (TTA)

Uses the 8 unique symmetries of the dihedral group D_4 (rotations + flips), averages predictions. Implemented with pure PyTorch tensor ops.

$$P_{\text{TTA}}(y=1|x) = (1/8) \cdot \sum_{\{g \in D_4\}} \sigma(f_{\theta}(g(x)))$$

15. Validation & Threshold Sweep

Computes Accuracy, Precision, Recall, F1, AUC-ROC. The optimal threshold τ^* is found via vectorised GPU sweep (91 thresholds) to maximise F1 on validation set.

16. Batch Normalisation Recalibration

After EMA/SWA, BN stats become stale. 25 batches with `momentum=0.0` overwrite them with current batch statistics, ensuring correct validation metrics.

17. Platt Scaling (Probability Calibration)

Fits a logistic regression model $\tilde{P}(y=1|z) = 1/(1+\exp(-(A \cdot z+B)))$ on validation set probabilities to improve calibration. Calibrator is saved as `calibrator.pkl`.

18. Three-Phase Training Loop

- **Phase 1 (5 epochs):** Head + cross-attention only. Backbones frozen.
- **Phase 2 (8 epochs):** Head + last blocks of each backbone (LR $\times 0.1$).
- **Phase 3 (27 epochs):** Full model, heavy augmentation, layer-wise LRs, linear warmup, cosine annealing with warm restarts, gradient accumulation (effective batch 128), SWA after epoch 30, early stopping patience 12.

19. AMP: Automatic Mixed Precision

Automatically selects bfloat16 (Ampere+) or float16+GradScaler (Volta/Turing) or fp32 (older GPUs). Saves memory and accelerates training.

20. Full Math Reference Index

Formula	Section	Used In
$\sigma(x)=1/(1+e^{-x})$	1.1	Binary head, losses
$GELU(x)=x\cdot\Phi(x)$	1.2	Projection layers
$FL = -(1-p_t)^{\gamma} \cdot BCE$	2.2	FocalLossWithLogits
ASL with $\gamma^+=1, \gamma^-=4$	2.3	AsymmetricLoss
Channel attention: $\sigma(MLP(Avg)+MLP(Max))$	3.1	CBAM
AttentionPool: $\sum \text{softmax}(s) \cdot x$	3.3	AttentionPool2d
Multi-Head Attention: $\text{softmax}(QK^T/\sqrt{d_k}) \cdot V$	3.4	Ensemble cross_attn
AdamW: $m_t=\beta_1m_{t-1}+(1-\beta_1)g_t$, weight decay decoupled	6.1	Optimizer
Cosine annealing: $\eta_{min}+\frac{1}{2}(\eta_{max}-\eta_{min})(1+\cos(\pi \cdot t/T_0))$	6.2	LR scheduler
$F1 = 2 \cdot \text{Prec} \cdot \text{Rec} / (\text{Prec} + \text{Rec})$	8.2	Threshold sweep objective
Platt: $1/(1+\exp(-(Az+B)))$	9	Probability calibration
D_4 TTA average over 8 orientations	11	_tta_predict

This guide covers every major class and function in the notebook with the full mathematical context from the reference PDF. Use your browser’s “Print” → “Save as PDF” to generate a beautiful, paginated document.